

easy-db GUI 界面工具开发指导书

一、模块概述

1.1 什么是 GUI 界面工具？

GUI (Graphical User Interface, 图形用户界面) 工具是通过可视化窗口与数据库进行交互的客户端程序。用户通过点击按钮、输入框、下拉菜单等图形控件来操作数据库，而不需要记忆命令或编写代码。

本次任务：为 `easy-db` 数据库开发一个 **桌面 GUI 客户端**，让用户可以通过图形界面完成所有数据库操作。

1.2 效果预览

easy-db GUI Client

[-] [□] [×]

连接设置

服务器: [127.0.0.1] 端口: [8080]

● 已连接

[连接]

[断开]

操作方式:

● 快捷操作

[○ 命令模式]

Key: [user:001] TTL: [] 秒

Value: [张三]

[SET]

[GET]

[DEL]

[EXISTS]

[KEYS]

查询结果:

{ "code": 200, "message": "OK", "data": "张三" }

命令模式:

> SET user:002 李四

OK

> GET user:002

"李四"

统计信息: 条目数: 5 | 命中率: 85% | 内存使用: 2.3MB

二、功能需求

2.1 核心功能（必做）

需求 2.1.1：连接管理

目标：用户可以通过界面配置并管理到服务端的连接。

功能点：

控件	说明
服务器地址输入框	输入服务端 IP 地址，默认 127.0.0.1
端口输入框	输入服务端端口，默认 8080
连接按钮	点击后连接到服务端
断开按钮	点击后断开当前连接
状态指示灯	显示连接状态（● 已连接 / ○ 未连接）

验收标准：

- ☐ 能成功连接到服务端
- ☐ 连接失败时弹出错误提示框
- ☐ 状态指示灯正确反映连接状态
- ☐ 断开连接后无法执行操作

需求 2.1.2：数据操作（快捷模式）

目标：通过按钮和输入框快速执行常用操作。

功能点：

操作	输入	输出
SET	Key、Value、TTL（可选）	显示 "OK"
GET	Key	显示查询到的值
DEL	Key	显示 "OK"
EXISTS	Key	显示 "true" 或 "false"
KEYS	Pattern（可选，默认 *）	显示键列表

界面设计：

Key: [_____]

TTL: [] 秒

Value: [_____]

[SET]

[GET]

[DEL]

[EXISTS]

[KEYS]

验收标准：

- ☐ 所有按钮点击后能正确执行对应的数据库操作
- ☐ 操作结果显示在结果区域
- ☐ 支持 TTL 过期时间设置（选做）

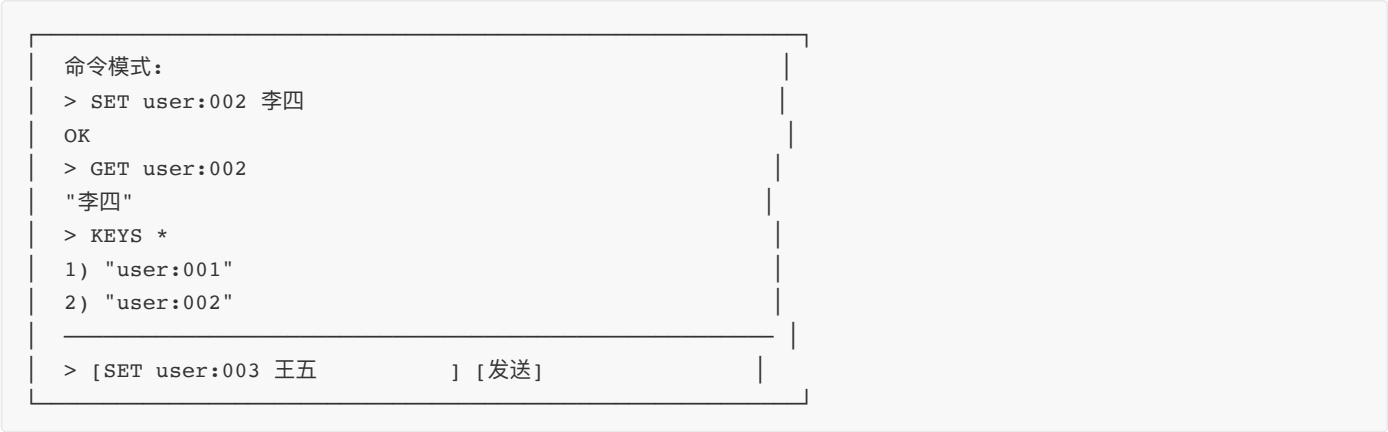
需求 2.1.3：命令模式

目标：用户可以直接输入原始命令，类似于 CLI 工具。

功能点：

- 提供命令行输入框
- 用户输入命令后按回车或点击发送按钮执行
- 命令结果按顺序显示在输出区域

界面设计：



验收标准：

- ☐ 能输入并执行任意命令
- ☐ 命令历史可滚动查看
- ☐ 支持上下键浏览历史命令（选做）

需求 2.1.4：结果展示

目标：清晰展示命令执行结果。

功能点：

- 使用文本区域展示结果
- 支持滚动查看历史结果
- 区分不同命令的输出（用分隔线或时间戳）

验收标准：

- ☐ 结果区域能正确显示所有类型的响应
- ☐ 长结果可滚动查看
- ☐ 结果展示不影响界面布局

2.2 增强功能（选做加分）

功能	说明	加分
数据表格展示	<code>KEYS</code> 结果显示为表格形式	+3
键值列表	左侧显示所有键，点击自动填充到输入框	+4
数据导出/导入	支持导出 JSON 文件，导入恢复数据	+3
深色模式	支持深色/浅色主题切换	+2
快捷键	支持 Ctrl+Enter 等快捷键	+2
多标签页	支持同时连接多个服务器	+4

三、技术选型

3.1 方案 A：Java Swing（推荐）

优点：

- JDK 内置，无需额外依赖
- 成熟稳定，文档丰富
- 学习曲线平缓

缺点：

- 界面相对老旧
- 复杂布局较难实现

示例：

```
import javax.swing.*;
import java.awt.*;

public class GuiClient extends JFrame {
    public GuiClient() {
        setTitle("easy-db GUI Client");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
        // ... 构建界面
    }
}
```

3.2 方案 B：JavaFX（进阶）

优点：

- 界面更现代
- 支持 CSS 样式
- 支持 FXML 声明式布局

缺点：

- JDK 8-10 内置，JDK 11+ 需要单独引入
- 学习曲线较陡

3.3 推荐选择

学生情况	推荐方案
没有 GUI 开发经验	使用 Swing ，重点学习基本组件和事件处理
有 GUI 开发经验或想挑战	使用 JavaFX ，制作更精美的界面

四、代码实现（Swing 示例）

4.1 包结构

```
src/main/java/
├── gui/                                ← 新增 GUI 模块
│   ├── GuiClient.java                # 主窗口
│   ├── ConnectionPanel.java          # 连接设置面板
│   ├── QuickOpsPanel.java            # 快捷操作面板
│   ├── ConsolePanel.java             # 命令模式面板
│   └── ResultPanel.java               # 结果展示面板
└── ...
```

4.2 完整实现

GuiClient.java

```
package gui;

import client.SocketClient;
import javax.swing.*;
import java.awt.*;
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;

/**
 * easy-db GUI 客户端主窗口
 */
public class GuiClient extends JFrame {

    private SocketClient client;
    private JLabel statusLabel;
    private JTextArea resultArea;
    private JTextField keyField;
    private JTextField valueField;
    private JTextField ttlField;
    private JTextField commandField;
    private JTextField hostField;
    private JTextField portField;
    private JButton connectBtn;
    private JButton disconnectBtn;
    private DefaultListModel<String> historyModel;

    public GuiClient() {
        setTitle("easy-db GUI Client");
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        setSize(900, 700);
        setLocationRelativeTo(null);

        // 添加窗口关闭事件
        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                closeClient();
            }
        });

        // 初始化组件
        initComponents();
        setVisible(true);
    }
}
```

```

private void initComponents() {
    setLayout(new BorderLayout(10, 10));

    // 顶部: 连接面板
    add(createConnectionPanel(), BorderLayout.NORTH);

    // 中间: 操作面板 + 结果面板
    JSplitPane mainSplit = new JSplitPane(JSplitPane.VERTICAL_SPLIT);
    mainSplit.setTopComponent(createOperationPanel());
    mainSplit.setBottomComponent(createResultPanel());
    mainSplit.setResizeWeight(0.4);
    add(mainSplit, BorderLayout.CENTER);

    // 底部: 状态栏
    add(createStatusBar(), BorderLayout.SOUTH);
}

/**
 * 创建连接面板
 */
private JPanel createConnectionPanel() {
    JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT, 10, 10));
    panel.setBorder(BorderFactory.createTitledBorder("连接设置"));

    hostField = new JTextField("127.0.0.1", 12);
    portField = new JTextField("8080", 6);
    connectBtn = new JButton("连接");
    disconnectBtn = new JButton("断开");
    disconnectBtn.setEnabled(false);
    statusLabel = new JLabel("○ 未连接");
    statusLabel.setForeground(Color.RED);

    // 连接按钮事件
    connectBtn.addActionListener(e -> connectToServer());

    // 断开按钮事件
    disconnectBtn.addActionListener(e -> disconnectFromServer());

    panel.add(new JLabel("服务器:"));
    panel.add(hostField);
    panel.add(new JLabel("端口:"));
    panel.add(portField);
    panel.add(connectBtn);
    panel.add(disconnectBtn);
    panel.add(Box.createHorizontalStrut(20));
    panel.add(statusLabel);

    return panel;
}

/**
 * 创建操作面板
 */
private JPanel createOperationPanel() {
    JPanel panel = new JPanel(new BorderLayout(5, 5));
    panel.setBorder(BorderFactory.createTitledBorder("操作"));

```

```

// Tab 切换: 快捷操作 / 命令模式
JTabbedPane tabbedPane = new JTabbedPane();
tabbedPane.addTab("快捷操作", createQuickOpsPanel());
tabbedPane.addTab("命令模式", createConsolePanel());
panel.add(tabbedPane, BorderLayout.CENTER);

return panel;
}

/**
 * 创建快捷操作面板
 */
private JPanel createQuickOpsPanel() {
    JPanel panel = new JPanel(new GridBagLayout());
    GridBagConstraints gbc = new GridBagConstraints();
    gbc.insets = new Insets(5, 5, 5, 5);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    // 第一行: Key
    gbc.gridx = 0; gbc.gridy = 0;
    panel.add(new JLabel("Key:"), gbc);
    gbc.gridx = 1; gbc.gridy = 0; gbc.weightx = 1.0;
    keyField = new JTextField(20);
    panel.add(keyField, gbc);

    // 第二行: Value
    gbc.gridx = 0; gbc.gridy = 1;
    gbc.weightx = 0;
    panel.add(new JLabel("Value:"), gbc);
    gbc.gridx = 1; gbc.gridy = 1; gbc.weightx = 1.0;
    valueField = new JTextField(20);
    panel.add(valueField, gbc);

    // 第三行: TTL
    gbc.gridx = 0; gbc.gridy = 2;
    gbc.weightx = 0;
    panel.add(new JLabel("TTL (秒):"), gbc);
    gbc.gridx = 1; gbc.gridy = 2; gbc.weightx = 1.0;
    ttlField = new JTextField("0", 5);
    panel.add(ttlField, gbc);

    // 第四行: 按钮
    JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.LEFT, 5, 0));
    JButton setBtn = new JButton("SET");
    JButton getBtn = new JButton("GET");
    JButton delBtn = new JButton("DEL");
    JButton existsBtn = new JButton("EXISTS");
    JButton keysBtn = new JButton("KEYS");

    // 设置按钮事件
    setBtn.addActionListener(e -> executeSet());
    getBtn.addActionListener(e -> executeGet());
    delBtn.addActionListener(e -> executeDel());
    existsBtn.addActionListener(e -> executeExists());
    keysBtn.addActionListener(e -> executeKeys());

    btnPanel.add(setBtn);
    btnPanel.add(getBtn);

```

```

        btnPanel.add(delBtn);
        btnPanel.add(existsBtn);
        btnPanel.add(keysBtn);

        gbc.gridx = 0; gbc.gridy = 3; gbc.gridwidth = 2;
        panel.add(btnPanel, gbc);

        return panel;
    }

    /**
     * 创建命令模式面板
     */
    private JPanel createConsolePanel() {
        JPanel panel = new JPanel(new BorderLayout(5, 5));

        // 命令历史显示区域
        historyModel = new DefaultListModel<>();
        JList<String> historyList = new JList<>(historyModel);
        historyList.setFont(new Font("Monospaced", Font.PLAIN, 13));
        JScrollPane historyScroll = new JScrollPane(historyList);
        historyScroll.setPreferredSize(new Dimension(0, 120));
        panel.add(historyScroll, BorderLayout.CENTER);

        // 命令输入区域
        JPanel inputPanel = new JPanel(new BorderLayout(5, 0));
        commandField = new JTextField();
        commandField.addActionListener(e -> executeCommand());
        JButton sendBtn = new JButton("发送");
        sendBtn.addActionListener(e -> executeCommand());

        inputPanel.add(commandField, BorderLayout.CENTER);
        inputPanel.add(sendBtn, BorderLayout.EAST);
        panel.add(inputPanel, BorderLayout.SOUTH);

        // 添加清空按钮到右上角
        JPanel topPanel = new JPanel(new BorderLayout());
        JButton clearBtn = new JButton("清空历史");
        clearBtn.addActionListener(e -> {
            historyModel.clear();
            resultArea.append(">>> 历史已清空\n");
        });
        topPanel.add(clearBtn, BorderLayout.EAST);
        panel.add(topPanel, BorderLayout.NORTH);

        return panel;
    }

    /**
     * 创建结果面板
     */
    private JPanel createResultPanel() {
        JPanel panel = new JPanel(new BorderLayout(5, 5));
        panel.setBorder(BorderFactory.createTitledBorder("结果"));

        resultArea = new JTextArea();
        resultArea.setEditable(false);
        resultArea.setFont(new Font("Monospaced", Font.PLAIN, 13));
    }

```



```

        resultArea.setBackground(new Color(240, 240, 240));
        JScrollPane scrollPane = new JScrollPane(resultArea);
        panel.add(scrollPane, BorderLayout.CENTER);

        // 清空按钮
        JPanel btnPanel = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        JButton clearBtn = new JButton("清空结果");
        clearBtn.addActionListener(e -> resultArea.setText(""));
        btnPanel.add(clearBtn);
        panel.add(btnPanel, BorderLayout.SOUTH);

        return panel;
    }

    /**
     * 创建状态栏
     */
    private JPanel createStatusBar() {
        JPanel panel = new JPanel(new FlowLayout(FlowLayout.LEFT));
        panel.setBackground(new Color(240, 240, 240));
        JLabel label = new JLabel("就绪");
        panel.add(label);
        return panel;
    }

    // ===== 业务逻辑 =====

    private void connectToServer() {
        try {
            String host = hostField.getText();
            int port = Integer.parseInt(portField.getText());

            client = new SocketClient(host, port);
            client.connect();

            statusLabel.setText("● 已连接 " + host + ":" + port);
            statusLabel.setForeground(Color.GREEN);
            connectBtn.setEnabled(false);
            disconnectBtn.setEnabled(true);
            resultArea.append(">>> 连接成功: " + host + ":" + port + "\n");

        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "连接失败: " + e.getMessage(), "错误",
JOptionPane.ERROR_MESSAGE);
            statusLabel.setText("○ 连接失败");
            statusLabel.setForeground(Color.RED);
        }
    }

    private void disconnectFromServer() {
        closeClient();
        statusLabel.setText("○ 已断开");
        statusLabel.setForeground(Color.RED);
        connectBtn.setEnabled(true);
        disconnectBtn.setEnabled(false);
        resultArea.append(">>> 已断开连接\n");
    }
}

```

```

private void closeClient() {
    if (client != null) {
        try {
            client.disconnect();
        } catch (Exception e) {
            // ignore
        }
        client = null;
    }
    dispose();
}

private void executeSet() {
    String key = keyField.getText();
    String value = valueField.getText();
    String ttlText = ttlField.getText();

    if (key.isEmpty() || value.isEmpty()) {
        showError("Key 和 Value 不能为空");
        return;
    }

    try {
        String result;
        if (!ttlText.isEmpty() && !ttlText.equals("0")) {
            result = client.sendCommand("SET " + key + " " + value + " " + ttlText);
        } else {
            result = client.sendCommand("SET " + key + " " + value);
        }
        resultArea.append(">>> SET " + key + " = " + value + "\n");
        resultArea.append(result + "\n");
    } catch (Exception e) {
        showError(e.getMessage());
    }
}

private void executeGet() {
    String key = keyField.getText();
    if (key.isEmpty()) {
        showError("Key 不能为空");
        return;
    }

    try {
        String result = client.sendCommand("GET " + key);
        resultArea.append(">>> GET " + key + "\n");
        resultArea.append(result + "\n");
    } catch (Exception e) {
        showError(e.getMessage());
    }
}

private void executeDel() {
    String key = keyField.getText();
    if (key.isEmpty()) {
        showError("Key 不能为空");
        return;
    }
}

```

```

    try {
        String result = client.sendCommand("DEL " + key);
        resultArea.append(">>> DEL " + key + "\n");
        resultArea.append(result + "\n");
    } catch (Exception e) {
        showError(e.getMessage());
    }
}

private void executeExists() {
    String key = keyField.getText();
    if (key.isEmpty()) {
        showError("Key 不能为空");
        return;
    }

    try {
        String result = client.sendCommand("EXISTS " + key);
        resultArea.append(">>> EXISTS " + key + "\n");
        resultArea.append(result + "\n");
    } catch (Exception e) {
        showError(e.getMessage());
    }
}

private void executeKeys() {
    String pattern = keyField.getText();
    if (pattern.isEmpty()) {
        pattern = "*";
    }

    try {
        String result = client.sendCommand("KEYS " + pattern);
        resultArea.append(">>> KEYS " + pattern + "\n");
        resultArea.append(result + "\n");
    } catch (Exception e) {
        showError(e.getMessage());
    }
}

private void executeCommand() {
    String command = commandField.getText().trim();
    if (command.isEmpty()) return;

    try {
        String result = client.sendCommand(command);
        resultArea.append(">>> " + command + "\n");
        resultArea.append(result + "\n");
        historyModel.addElement("> " + command);
        commandField.setText("");
    } catch (Exception e) {
        showError(e.getMessage());
    }
}

private void showError(String message) {
    JOptionPane.showMessageDialog(this, message, "错误", JOptionPane.ERROR_MESSAGE);
}

```

```

    }

    // ===== 主方法 =====

    public static void main(String[] args) {
        // 使用 Swing 的线程安全方式启动
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception e) {
                // ignore
            }
            new GuiClient();
        });
    }
}

```

五、使用示例

5.1 启动程序

```

# 方式一：直接运行 Java 类
$ java -cp target/easy-db-1.0-SNAPSHOT.jar gui.GuiClient

# 方式二：打包为 jar 后运行
$ java -jar easy-db-gui.jar

```

5.2 操作演示

1. 连接服务端

在连接面板输入服务器地址和端口，点击"连接"：

```
>>> 连接成功：127.0.0.1:8080
```

2. 存储数据（快捷模式）

在 Key 输入框输入 `user:001`，Value 输入框输入 `张三`，点击 **SET**：

```
>>> SET user:001 = 张三
OK
```

3. 查询数据

输入 Key 后点击 **GET**：

```
>>> GET user:001
"张三"
```

4. 使用命令模式

在命令模式输入框输入 `KEYS *`，按回车：

```
>>> KEYS *
1) "user:001"
2) "user:002"
```

5. 完整操作截图模拟

easy-db GUI Client

[-] [□] [×]

连接设置

服务器: [127.0.0.1] 端口: [8080]

● 已连接

[连接]

[断开]

操作:

● 快捷操作

[○ 命令模式]

Key: [user:001] TTL: [60] 秒

Value: [张三]

[SET]

[GET]

[DEL]

[EXISTS]

[KEYS]

结果:

>>> 连接成功: 127.0.0.1:8080

>>> SET user:001 = 张三

OK

>>> GET user:001

"张三"

>>> KEYS *

1) "user:001"

2) "user:002"

>>> EXISTS user:001

true

命令模式:

> SET user:003 王五

OK

> GET user:003

"王五"

> [SET user:004 赵六] [发送]

统计: 条目数: 4 | 命中率: 100% | 内存使用: 2.3MB

六、开发任务清单

阶段	任务	状态
阶段一	创建 <code>gui</code> 包和 <code>GuiClient</code> 主窗口类	☐
阶段二	实现连接面板（地址、端口、连接/断开按钮）	☐
阶段三	实现快捷操作面板（Key/Value/TTL + 5个按钮）	☐
阶段四	实现结果展示区域（文本区域 + 滚动条）	☐
阶段五	实现命令模式面板（历史 + 输入框 + 发送）	☐
阶段六	实现所有按钮的事件监听和业务逻辑	☐
阶段七	实现状态栏显示	☐
阶段八（选做）	数据表格展示 <code>KEYS</code> 结果	☐
阶段九（选做）	左侧键值列表，点击自动填充	☐
阶段十（选做）	数据导出/导入功能	☐

七、常见问题

Q1：Swing 界面卡顿、无响应？

A：不要在事件处理线程中执行耗时操作。如果 `sendCommand()` 可能耗时，应该放在后台线程：

```
new Thread(() -> {
    String result = client.sendCommand(command);
    SwingUtilities.invokeLater(() -> resultArea.append(result));
}).start();
```

Q2：如何让窗口居中显示？

A：

```
setLocationRelativeTo(null);
```

Q3：如何让文本框默认聚焦？

A：

```
commandField.requestFocusInWindow();
```

Q4：Swing 组件中文显示乱码？

A：设置字体支持中文：

```
resultArea.setFont(new Font("微软雅黑", Font.PLAIN, 13));
```

Q5：打包 jar 后无法运行？

A：确保 `MANIFEST.MF` 中指定了主类：

```
Main-Class: gui.GuiClient
```

Q6：如何让界面更美观？

A：

- 使用 `UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName())` 使用系统样式
- 使用 `FlatLaf` 等第三方主题库

八、提交要求

1. 代码：GUI 模块全部源码
2. 打包：提供可执行的 `easy-db-gui.jar`
3. 文档：在课程设计报告中增加 **GUI 客户端** 章节，内容包括：
 - 界面布局设计图
 - 各功能模块说明
 - 使用示例截图（至少 5 张）
 - 布局管理器选择原因
4. 演示：验收时现场演示 GUI 工具的所有功能

九、参考资料

- [Java Swing 教程 \(Oracle\)](#)
- [Swing 布局管理器](#)
- [Java 事件处理机制](#)

祝同学们顺利完成 GUI 界面工具开发！